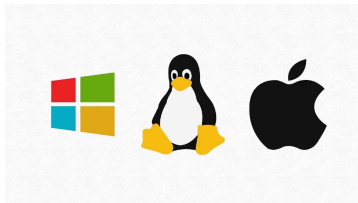


컴퓨터 시스템

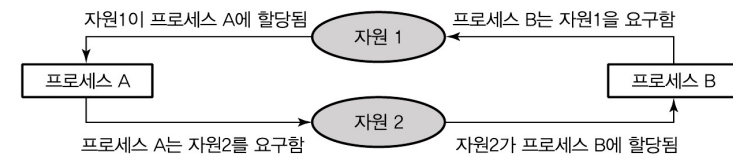
Deadlock(교착상태)



This material is reproduced based on
핵심 컴퓨터구조 및 운영체제 해설(최종연저, 예문사),
쉽게 배우는 운영체제(조성호, 한빛아카데미, 2018)
Copyrights are reserved by corresponding authors.

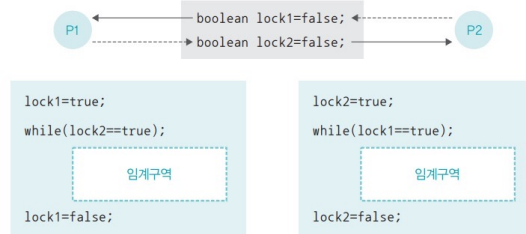
교착 상태의 정의

- 교착 상태
 - 2개 이상의 프로세스가 다른 프로세스의 작업이 끝나기만 기다리며 작업을 더 이상 진행하지 못하는 상태
- 아사 상태와 차이점
 - 아사 현상 : 운영체제가 잘못된 정책을 사용하여 특정 프로세스의 작업이 지연되는 문제
 - 교착 상태 : 여러 프로세스가 작업을 진행하다 보니 자연 발생적으로 일어나는 문제



교착 상태의 발생

- 공유 변수
 - 교착 상태는 공유 변수를 사용할 때 발생



- 응용 프로그램
 - 데이터베이스 같은 응용 프로그램에서도 교착 상태 발생
 - 데이터베이스는 데이터의 일관성을 유지하기 위해 잠금을 사용하는데, 이때 교착 상태가 발생할 수 있음

교착상태 원인

원인	설명
상호 배제 (Mutual Exclusion)	최소한 하나의 자원이 비공유 모드로 점유되어야 한다. 비공유 모드에서는 한 번에 한 프로세스만이 그 자원을 사용할 수 있다. 다른 프로세스가 그 자원을 요청하면 요청 프로세스는 자원이 방출될 때까지 지연된다.
점유하며 대기 (Hold-and-wait)	최소한 하나의 자원을 점유한 채 현재 다른 프로세스에 의해 점유된 자원을 추가로 얻기 위해 대기하고 있는 프로세스가 반드시 있어야 한다.
비선점 (Nonpreemption)	자원들을 선점할 수 없어야 한다. 즉 자원이 강제로 방출될 수 없고 점유하고 있는 프로세스가 태스크를 종료한 후 그 프로세스에 의해 자발적으로만 방출될 수 있다.
환형 대기 (Circular wait)	대기하고 있는 프로세스의 집합 {P1...Pn}에서 P1은 P2가 점유한 자원에 대기하고 P2는 P3가 점유한 자원을 대기하고 ... Pn-1은 Pn이 점유한 자원을 대기하며 Pn은 P1이 점유한 자원을 대기한다.

식사하는 철학자 문제와 교착 상태 필요조건

- 상호 배제 : 포크는 한 사람이 사용하면 다른 사람이 사용할 수 없는 배타적인 자원임
- 비선점 : 철학자 중 어떤 사람의 힘이 월등하여 옆 사람의 포크를 빼앗을 수 없음
- 점유와 대기 : 한 철학자가 두 자원(왼쪽 포크와 오른쪽 포크)을 다 점유하거나, 반대로 두 자원을 다 기다릴 수 없음
- 원형 대기 : 철학자들은 동그런 식탁에서 식사를 함, 원을 이룬다는 것은 선후 관계를 결정할 수 없어 문제가 계속 맴돈다는 의미(사각형 식탁에서 한 줄로 앉아서 식사를 한다면 교착 상태가 발생하지 않음)

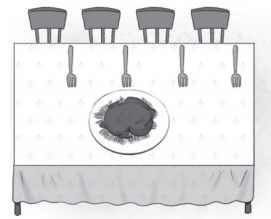


그림 6-10 사각형 식탁과 식사하는 철학자 문제

© 2021 Computer System

교착 상태 해결 방법

- 교착 상태 예방(prevention) : 교착 상태를 유발하는 네 가지 조건이 발생하지 않도록 무력화하는 방식으로 교착상태 조건 4가지에 대하여 각각의 방법이 존재 함.
- 교착 상태 회피(avoidance) : 교착상태가 발생하지 않도록 자원 할당량을 조절하여 교착 상태를 회피하는 방식
- 교착 상태 검출(detection)과 회복(recovery) : 교착 상태 검출은 어떤 제약을 가하지 않고 자원 할당 그래프를 모니터링하면서 교착 상태가 발생하는지 살펴보는 방식으로 만약 교착 상태가 발생하면 교착 상태 회복 단계가 진행됨

© 2021 Computer System

교착상태 예방

해결방안	설명		
예방	교착상태가 발생하려면 4가지의 필요조건이 만족되어야 하므로 이들 조건 중 최소한 하나가 성립하지 않도록 보장함으로써 교착상태를 예방할 수 있다.		
	구분	특징	단점
	상호 배제의 부정	공유할 수 없는 자원을 사용할 때 성립	자원낭비 및 비용증가
	점유하며 대기 부정	프로세스는 필요한 모든 자원을 한 번에 요청하고 시스템은 요청된 자원을 전부 할당 또는 전혀 할당하지 않는 방식	• 자원낭비 및 비용증가 • 자원공유 불가능 • Starvation 발생가능
	비선점 부정	어떤 자원을 가진 프로세스가 더 이상 자원할당 요구가 받아지지 않으면 점유 자원을 반납	• 비용증가 • Starvation 발생가능 • 일부 자원은 안전하게 선점 불가
	원형 대기 부정	모든 프로세스에게 각 자원의 유형별로 할당순서를 부여하는 방법	새로운 자원 추가시 재구성 필요

© 2021 Computer System

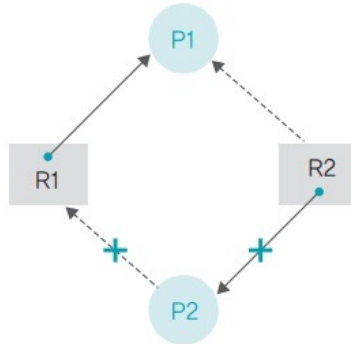
교착 상태 예방

- 교착상태 조건 4가지에 대하여 각각의 방식이 존재
- 상호 배제 예방
 - 시스템 내에 있는 상호 배타적인 모든 자원, 즉 독점적으로 사용할 수 있는 자원을 없애 버리는 방법
 - 현실적으로는 모든 자원을 공유할 수 없으며 상호 배제를 적용하여 보호해야 하는 자원이 있음
 - 상호 배제를 무력화하는 것은 사실상 어려움
- 비선점 예방
 - 모든 자원을 빼앗을 수 있도록 만드는 방법
 - 그러나 아사 현상을 일으켜 비선점 조건을 무력화하기는 어려움

© 2021 Computer System

교착 상태 예방

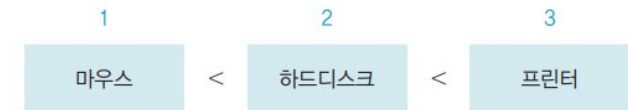
- 점유와 대기 예방
 - 프로세스가 자원을 점유한 상태에서 다른 자원을 기다리지 못하게 하는 방법
 - '전부 할당하거나 아니면 아예 할당하지 않는' 방식을 적용
 - 자원이 아닌 프로세스의 자원 사용 방식을 변화시켜 교착 상태를 처리한다는 점에서 의미가 있음
- 점유와 대기 예방의 단점
 - 프로세스가 자신이 사용하는 모든 자원을 자세히 알기 어려움
 - 자원의 활용성이 떨어짐
 - 많은 자원을 사용하는 프로세스가 적은 자원을 사용하는 프로세스보다 불리함
 - 결국 일괄 작업 방식으로 동작



© 2021 Computer System

교착 상태 예방

- 원형 대기 예방
 - 점유와 대기를 하는 프로세스들이 원형을 이루지 못하도록 막는 방법
 - 모든 자원에 숫자를 부여하고 숫자가 큰 방향으로만 자원을 할당하는 것
 - [예] 마우스를 할당받은 상태에서 프린터를 할당받을 수는 있지만 프린터를 할당받은 상태에서는 마우스나 하드디스크를 할당받을 수 없음
- 원형 대기 예방의 단점
 - 프로세스 작업 진행에 유연성이 떨어짐
 - 자원의 번호를 어떻게 부여할 것인가가 문제



© 2021 Computer System

교착 상태 예방

- 교착 상태 예방 정리
 - 교착 상태를 유발하는 네 가지 조건이 일어나지 않도록 제약을 가하는 방법
 - 자원을 보호하기 위해 상호 배제와 비선점을 예방하기 어려움
 - 점유와 대기, 원형 대기는 프로세스 작업 방식을 제한하고 자원을 낭비하기 때문에 사용할 수 없음

© 2021 Computer System

교착 상태 회피

- 교착 상태 회피의 개념
 - 프로세스에 자원을 할당할 때 어느 수준 이상의 자원을 나누어 주면 교착 상태가 발생하는지 파악하여 그 수준 이하로 자원을 나누어 주는 방법
 - 교착 상태가 발생하지 않는 범위 내에서만 자원을 할당하고, 교착 상태가 발생하는 범위에 있으면 프로세스를 대기시킴
 - 즉, 할당되는 자원의 수를 조절하여 교착 상태를 피함

© 2021 Computer System

교착 상태 회피

■ 안정 상태와 불안정 상태

- 교착 상태 회피는 자원의 총수와 현재 할당된 자원의 수를 기준으로 시스템을 안정 상태safe state와 불안정 상태unsafe state로 나누고 시스템이 안정 상태를 유지하도록 자원을 할당
- 할당된 자원이 적으면 안정 상태가 크고, 할당된 자원이 늘어날수록 불안정 상태가 커짐
- 교착 상태는 불안정 상태의 일부분이며, 불안정 상태가 커질수록 교착 상태가 발생할 가능성이 높아짐
- 교착 상태 회피는 안정 상태를 유지할 수 있는 범위 내에서 자원을 할당함으로써 교착 상태를 피함

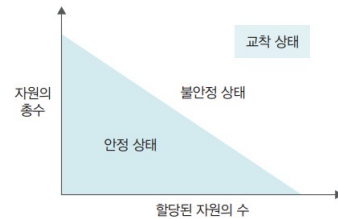
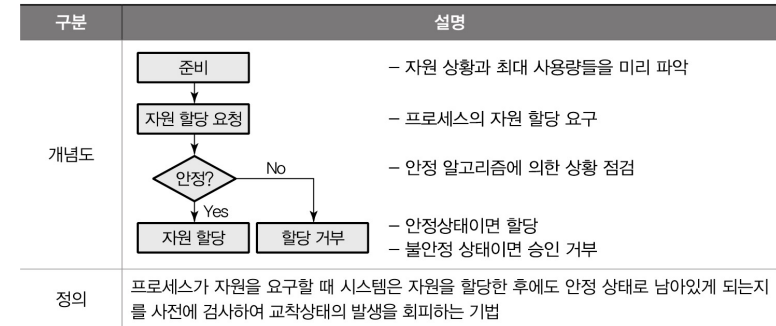


그림 6-15 안정 상태와 불안정 상태

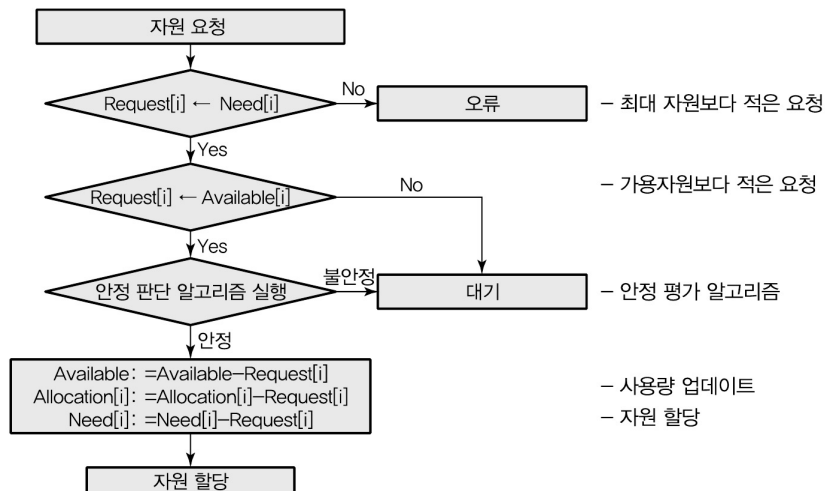
교착 상태 회피

■ 은행원 알고리즘

- 교착 상태 회피를 구현하는 대표적인 알고리즘
- 은행이 대출을 해주는 방식, 즉 대출 금액이 대출 가능한 범위 내이면(안정 상태이면) 허용되지만 그렇지 않으면 거부되는 것과 유사한 방식



은행가 알고리즘의 자원요청 처리 알고리즘



교착 상태 회피

■ 은행원 알고리즘에서 자원 할당 기준

- 각 프로세스의 기대 자원과 비교하여 가용 자원이 하나라도 크거나 같으면 자원을 할당
- 가용 자원이 어떤 기대 자원보다 크지 않으면 할당하지 않음

표 6-2 은행원 알고리즘의 변수

변수	설명
전체 자원(Total)	시스템 내 전체 자원의 수
가용 자원(Available)	시스템 내 현재 사용할 수 있는 자원의 수(가용 자원=전체 자원-모든 프로세스의 할당 자원)
최대 자원(Max)	각 프로세스가 선언한 최대 자원의 수
할당 자원(Allocation)	각 프로세스에 현재 할당된 자원의 수
기대 자원(Expect)	각 프로세스가 앞으로 사용할 자원의 수(기대 자원=최대 자원-할당 자원)

은행가 알고리즘 예제

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	1	1	2
P2	6	1	3	5	1	1			
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

P2 requests (1, 0, 1)

© 2021 Computer System

은행가 알고리즘 예제

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	0	1	1
P2	6	1	3	6	1	2			
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

© 2021 Computer System

은행가 알고리즘 예제

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	1	0	0	6	2	3
P2	0	0	0	0	0	0			
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	0	0	0	0	0	0	7	2	3
P2	0	0	0	0	0	0			
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

© 2021 Computer System

은행가 알고리즘 예제

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	0	0	0	0	0	0	9	3	4
P2	0	0	0	0	0	0			
P3	0	0	0	0	0	0			
P4	4	2	2	0	0	2			

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	0	0	0	0	0	0	9	3	6
P2	0	0	0	0	0	0			
P3	0	0	0	0	0	0			
P4	0	0	0	0	0	0			

© 2021 Computer System

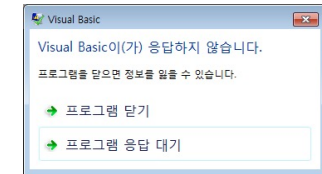
은행가 알고리즘 예제

	Claimed			Allocated			Available		
	R1	R2	R3	R1	R2	R3	R1	R2	R3
P1	3	2	2	2	0	1	0	1	1
P2	6	1	3	5	1	1			
P3	3	1	4	2	1	1			
P4	4	2	2	0	0	2			

© 2021 Computer System

교착 상태 검출

- 교착 상태 검출의 개념
 - 운영체제가 프로세스의 작업을 관찰하면서 교착 상태 발생 여부를 계속 주시하는 방식
 - 교착 상태가 발견되면 이를 해결하기 위해 교착 상태 회복 단계를 밟음
- 타임아웃을 이용한 교착 상태 검출
 - 일정 시간 동안 작업이 진행되지 않은 프로세스를 교착 상태가 발생한 것으로 간주하여 처리하는 방법
 - 교착 상태가 자주 발생하지 않을 것이라는 가정하에 사용하는 것으로, 특별한 알고리즘이 없어 쉽게 구현할 수 있음
 - 타임아웃이 되면 프로세스가 종료됨



© 2021 Computer System

교착 상태 검출

- 데이터베이스에서 타임아웃의 문제
 - 데이터베이스에서 타임아웃으로 프로세스가 종료되면 일부 데이터의 일관성이 깨질 수 있음
 - 데이터의 일관성이 깨지는 문제를 해결하기 위해 체크포인트와 롤백 사용
 - 체크포인트 : 작업을 하다가 문제가 발생하면 저장된 상태로 돌아오기 위한 표시
 - 롤백 : 작업을 하다가 문제가 발생하여 과거의 체크포인트로 되돌아가는 것

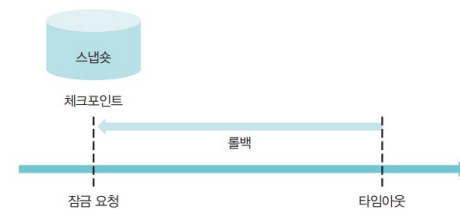


그림 6-20 체크포인트와 롤백

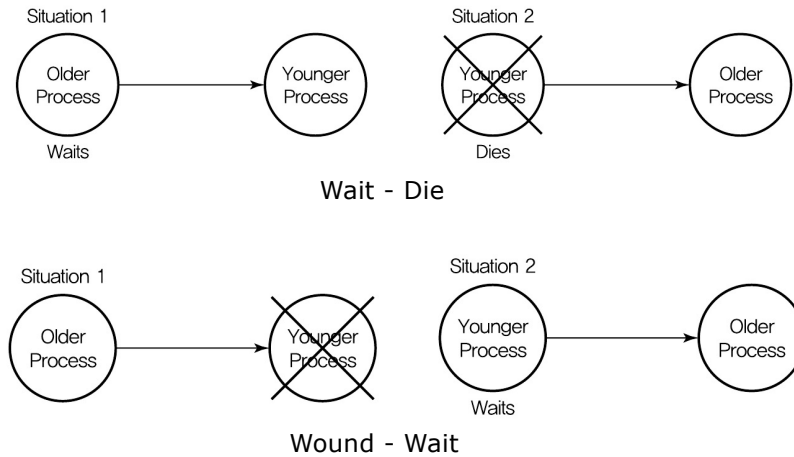
© 2021 Computer System

교착 상태 회복

- 교착 상태 회복
 - 교착 상태가 검출된 후 교착 상태를 푸는 후속 작업을 하는 것
 - 교착 상태 회복 단계에서는 교착 상태를 유발한 프로세스를 강제로 종료
- 프로세스를 강제로 종료하는 방법
 - ① 교착 상태를 일으킨 모든 프로세스를 동시에 종료
 - ② 교착 상태를 일으킨 프로세스 중 하나를 골라 순서대로 종료
 - 우선순위가 낮은 프로세스를 먼저 종료
 - 우선순위가 같은 경우 작업 시간이 짧은 프로세스를 먼저 종료
 - 위의 두 조건이 같은 경우 자원을 많이 사용하는 프로세스를 먼저 종료

© 2021 Computer System

교착 회피



© 2021 Computer System

자원 할당 그래프

■ 자원 할당 그래프

- 프로세스가 어떤 자원을 사용 중이고 어떤 자원을 기다리고 있는지를 방향성이 있는 그래프로 표현한 것
- 프로세스는 원으로, 자원은 사각형으로 표현



© 2021 Computer System

다중 자원과 사이클

- 다중 자원과 사이클
 - 다중 자원이 포함된 자원 할당 그래프에서는 대기 그래프와 그래프 감소 방법을 이용하여 사이클을 찾음
- 대기 그래프
 - 자원 할당 그래프에서 프로세스와 프로세스 간에 기다리는 관계만 나타낸 그래프
- 그래프 감소
 - 대기 그래프에서 작업이 끝날 가능성이 있는 프로세스의 화살표와 관련 프로세스의 화살표를 연속적으로 지워가는 작업

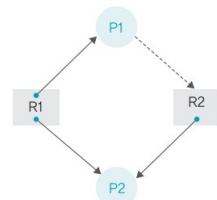


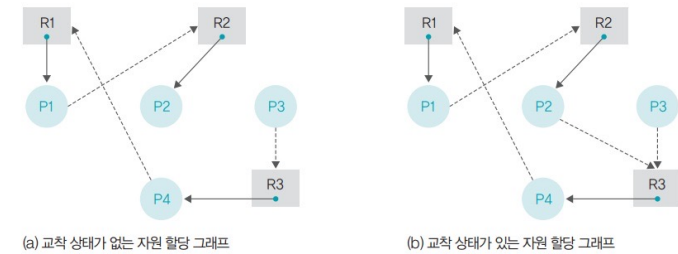
그림 6-23 다중 자원과 사이클

© 2021 Computer System

교착 상태 검출

■ 자원 할당 그래프를 이용한 교착 상태 검출

- 단일 자원을 사용하는 경우 자원 할당 그래프에 사이클 있으면 교착 상태



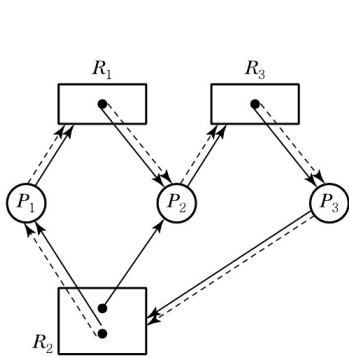
(a) 교착 상태가 없는 자원 할당 그래프

(b) 교착 상태가 있는 자원 할당 그래프

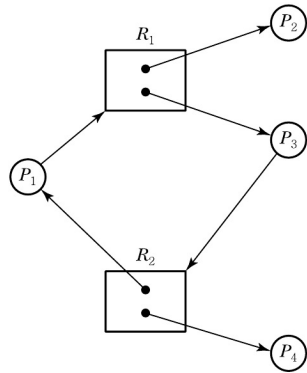
그림 6-22 자원 할당 그래프와 교착 상태

© 2021 Computer System

자원 할당 그래프



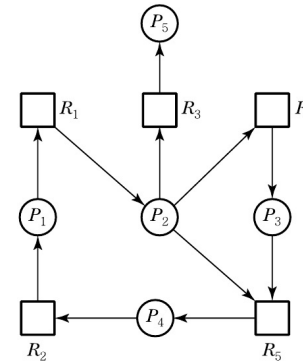
자원 할당 그래프 - 교착상태



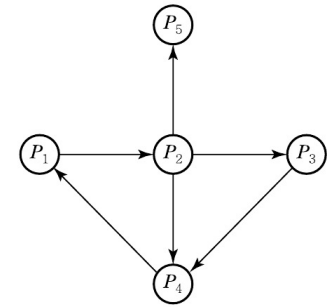
자원 할당 그래프 - 일반상태

© 2021 Computer System

자원할당 그래프 - 대기 그래프



자원 할당 그래프



대기 그래프

© 2021 Computer System

Summary



Image from <http://www.promotionalpro.com/blog/>

**Don't Guess....
Just ASK !**

© 2021 Computer System